



程序的基本结构



程序的控制结构

■学习目标

- 了解程序的基本结构和流程图的绘制
- 掌握程序的分支结构（ if 语句 ）
- 掌握程序的循环结构（ for语句和while语句 ）



程序的基本结构

- 程序由三种基本结构组成：
 - **顺序结构**：程序按照线性顺序依次执行，是程序的基础
 - **分支结构**：根据条件判断结果选择不同的**向前**执行路径
 - **循环结构**：根据条件判断结果**向后**反复执行
- 三种基本结构都有一个**入口**和一个**出口**
- 任何程序都由这三种基本结构组合而成

程序流程图

- 用一系列图形、流程线和文字说明描述程序的基本操作和控制流程
- 是程序分析和过程描述的最基本方式
- 包括 7 种基本元素



起止框



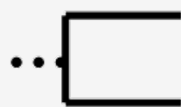
判断框



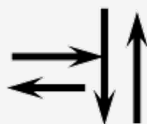
处理框



输入/输出框



注释框



流向线



连接点

程序流程图

判断一个条件是否成立，
并根据判断结果选择不同的
执行路径

表示一组处理过程

表示一个程序的
开始和结束

表示数据的输入
和结果的输出

起止框

判断框

处理框

输入/输出框

注释框

流向线

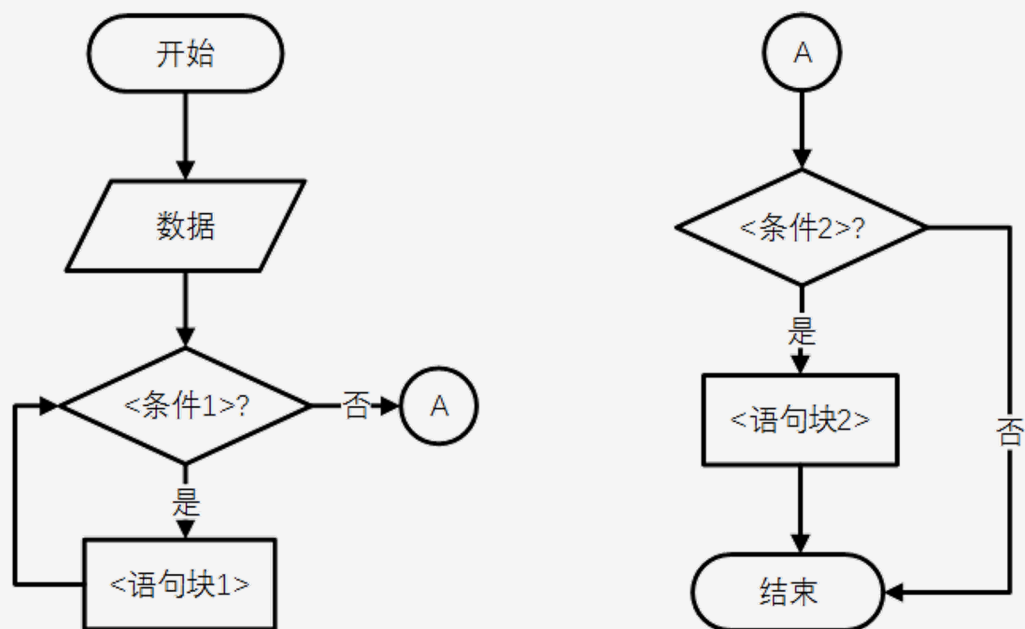
连接点

增加程序的解释

以带箭头的直线或曲线形
式指示程序的执行路径

将多个组成较大流程图的
若干部分连接到一起

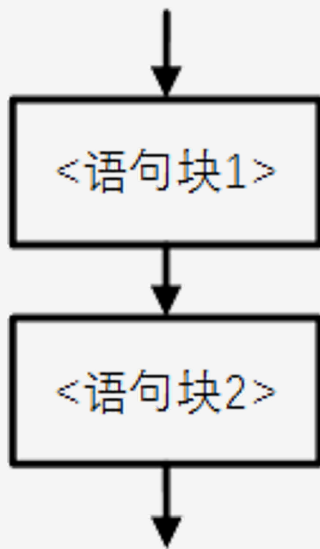
程序的流程图



程序流程图示例：由连接点 **A** 连接的一个程序

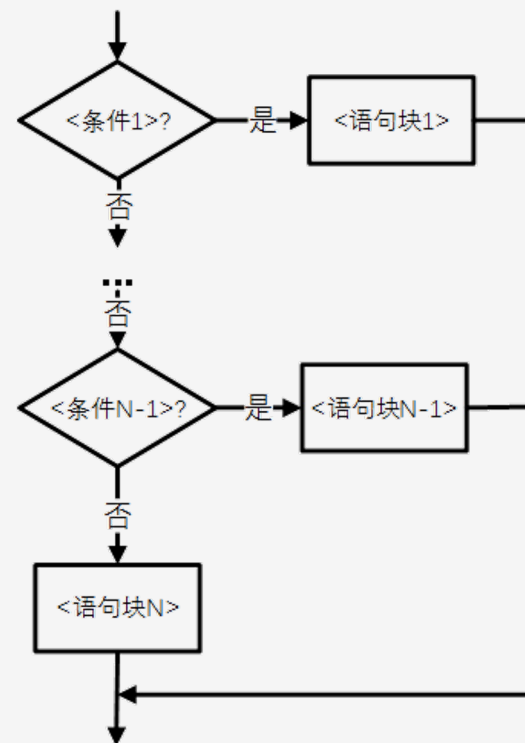
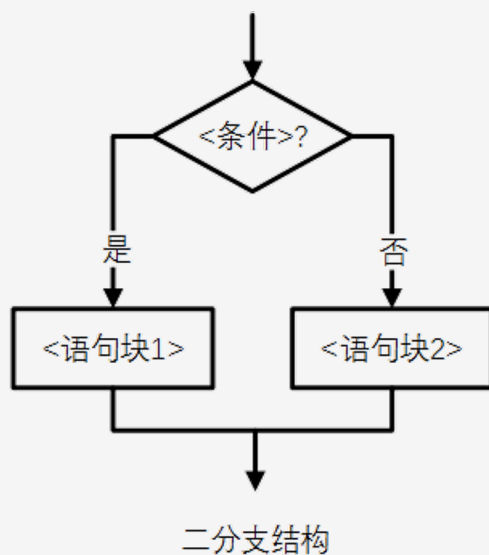
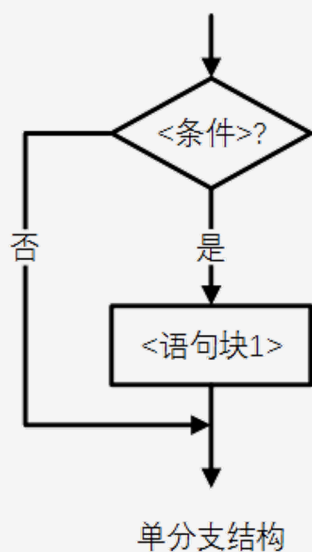
顺序结构

- 顺序结构是程序按照**线性顺序**依次执行的一种运行方式，其中语句块1和语句块2表示一个或一组顺序执行的语句



分支结构

- 分支结构是程序**根据条件判断结果**而选择不同向前执行路径的一种运行方式
- 包括**单分支结构**、**二分支结构**和**多分支结构**



分支结构

■ 单分支结构、二分支结构和多分支结构

单分支结构对应 `if` : 二分支结构对应 `if-else` : 多分支结构对应 `if-elif-else` :

```
python
```

```
if 条件:  
    语句块1
```

```
python
```

```
if 条件:  
    语句块1  
else:  
    语句块2
```

```
python
```

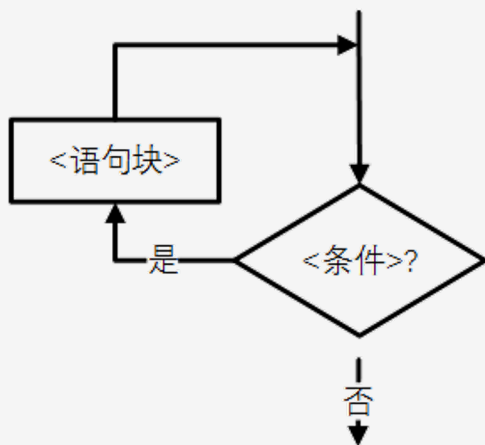
```
if 条件1:  
    语句块1  
elif 条件2:  
    语句块2  
...  
elif 条件N-1:  
    语句块N-1  
else:  
    语句块N
```

循环结构

- 循环结构：程序根据条件判断结果向后反复执行的一种运行方式
- 根据循环体触发条件不同，包括条件循环和遍历循环结构

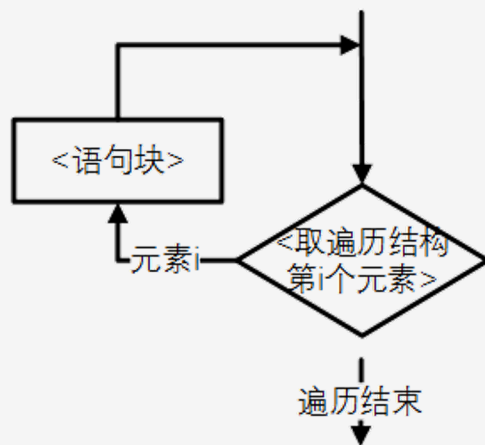


被反复执行的程序



条件循环

while语句



遍历循环

for语句

程序的基本结构实例

- 对于一个计算问题，可以用 **IPO**描述、**流程图**描述或者直接以**Python代码**方式描述

微实例4.1：圆面积和周长的计算

输入：圆半径R

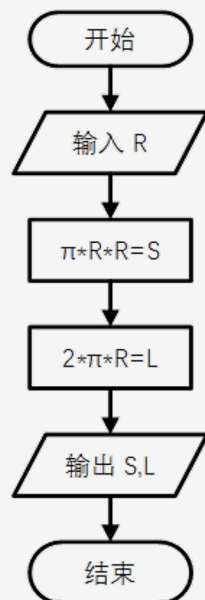
处理：

圆面积： $S = \pi * R * R$

圆周长： $L = 2 * \pi * R$

输出：圆面积S、周长L

(a) 问题IPO描述



(b) 流程图描述

```
1  R = float(input("请输入圆半径:"))
2  S = 3.1415 * R * R
3  L = 2 * 3.1415 * R
4  print("面积和周长:", S, L)
```

(c) Python代码描述

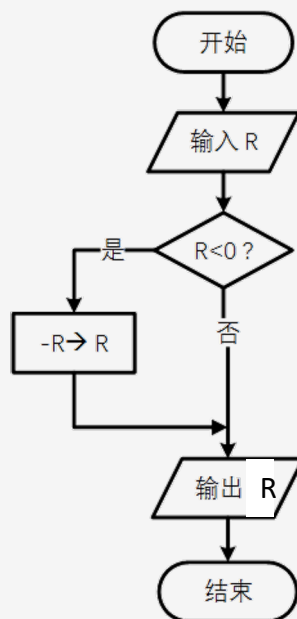
程序的基本结构实例

微实例4.2：实数绝对值的计算

输入：实数R

$$\text{处理: } |R| = \begin{cases} R & R \geq 0 \\ -R & R < 0 \end{cases}$$

输出：输出|R|



```
R = float(input("输入实数:"))
if R < 0:
    R = -R
print("绝对值: ", R)
```

(a) 问题IPO描述

(b) 流程图描述

(c) Python代码描述

程序分支结构

Branch structures of python procedure

单分支结构: if语句

- Python中 if 语句的语法格式：

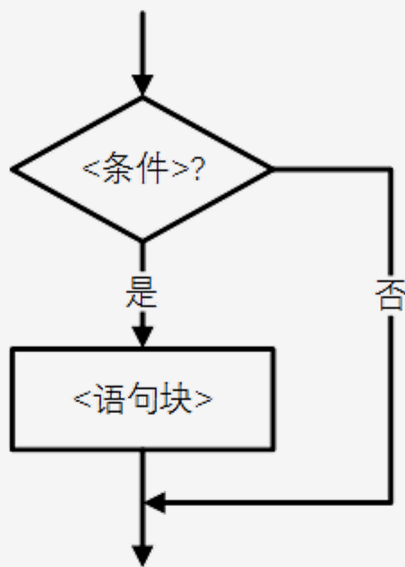
if <条件>:

语句块

- 语句块是 if 条件满足后执行的一个或多个语句序列
- 语句块中语句通过与if所在行形成缩进表达包含关系
- if 语句首先评估 <条件> 的结果值，如果结果为True，则执行语句块里的语句序列，然后控制转向程序的下一条语句。如果结果为 False，语句块里的语句会被跳过

单分支结构: if语句

- if 语句中语句块执行与否依赖于条件判断。但无论什么情况，之后控制都会转到 if 语句后与该语句同级别的下一条语句



if 语句的控制流程图

单分支结构: if语句

- if 中<条件>部分可以使用任何能够产生 True 或 False的语句
- 形成判断条件最常见的方式是采用关系操作符
- Python语言共有 6 个关系操作符

操作符	数学符号	操作符含义
<	<	小于
<=	≤	小于等于
>=	≥	大于等于
>	>	大于
==	=	等于
!=	≠	不等于

⚠ Python中" =" 表示赋值, " == " 表示等于

二分支结构: if-else语句

- Python中 **if-else** 语句用来形成二分支结构，语法格式如下：

- ```
if <条件>:
 <语句块1>
else:
 <语句块2>
```



## 二分支结构: if-else语句

- 二分支结构还有一种更简洁的表达方式，适合通过判断返回特定值，语法格式如下：

<表达式1> if <条件> else <表达式2>

如果 if 后<条件>满足，则返回<表达式1>，  
否则返回<表达式2>

```
PM = eval(input("请输入PM2.5数值: "))
print("空气{}污染!".format("存在" if PM >= 75 else "没有"))
```

```
n = int(input("请输入整数: "))
print(f"{n}是" + ("偶数" if n % 2 == 0 else "奇数"))
```

## 多分支结构: if-elif-else语句

- Python的 **if-elif-else** 描述多分支结构，语句格式如下：

if <条件1>:

    <语句块1>

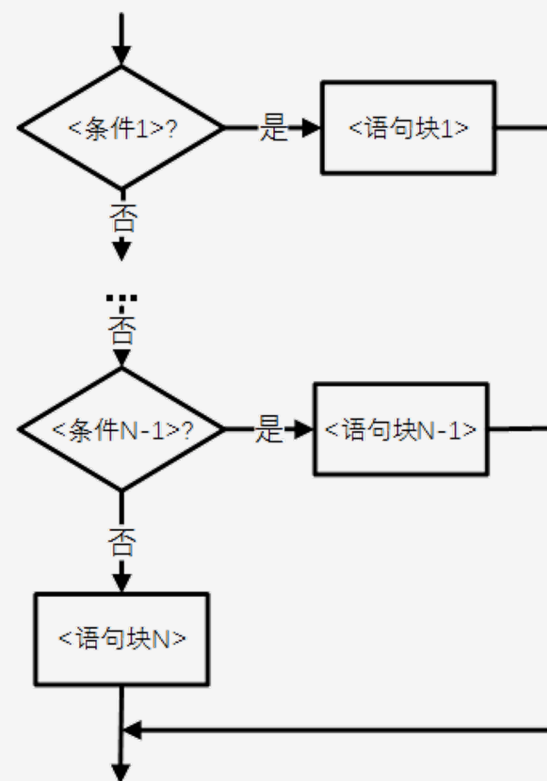
elif <条件2>:

    <语句块2>

...

else:

    <语句块N>





## 多分支结构: if-elif-else语句

---

- 多分支结构是**二分支结构的扩展**，这种形式通常用于设置同一个判断条件的多条执行路径
- Python依次寻找第一个结果为True的条件，执行该条件下的语句块，结束后跳过整个if-elif-else结构，执行后面的语句。如果没有任何条件成立，else下面的语句块被执行





## 多分支结构: if-elif-else语句

---

- 多分支结构是**二分支结构的扩展**，这种形式通常用于设置同一个判断条件的多条执行路径
- Python依次寻找第一个结果为True的条件，执行该条件下的语句块，结束后跳过整个if-elif-else结构，执行后面的语句。如果没有任何条件成立，else下面的语句块被执行



# 多分支结构: if-elif-else语句

## ■ 多个独立 if 与 if-elif-else 的区别

```
PM = eval(input("请输入PM2.5数值: "))
if 0 <= PM < 35:
 print("空气优质, 快去户外运动!")
if 35 <= PM < 75:
 print("空气良好, 适度户外活动! ")
if 75 <= PM:
 print("空气污染, 请小心! ")
```

⚠ 每个 if 独立判断, Python 会把三个条件全部检查一遍, 即使第一个已经命中了, 后面两个仍然要执行判断

```
PM = eval(input("请输入PM2.5数值: "))
if 0 <= PM < 35:
 print("空气优质, 快去户外运动!")
elif 35 <= PM < 75:
 print("空气良好, 适度户外活动! ")
else:
 print("空气污染, 请小心! ")
```

⚠ 命中第一个条件后, 后面的 elif/else 直接跳过, 不再判断, 因而 if-elif-else 效率更高, 命中即停

## ■ match-case 语句

- match-case 是 Python 3.10 引入的结构化模式匹配语法
- 适用于需要对一个值进行多种情况判断的场景
- 基本语法格式：

match 表达式：

```
case 模式1:
 语句块1
```

```
case 模式2:
 语句块2
```

```
...
```

```
case _:
 默认语句块
```

- match 后接要匹配的表达式 (能求值出一个结果就行)
- 每个 case 后接一个模式
- \_ 为通配符，匹配任意值，相当于 default
- 从上到下依次匹配，命中第一个即执行对应语句块

## ■ match-case 语句

- 示例——HTTP 状态码判断：

```
status = 404
```

```
match status:
```

```
 case 200:
```

```
 print("请求成功")
```

```
 case 301:
```

```
 print("永久重定向")
```

```
 case 404:
```

```
 print("页面未找到")
```

输出： 页面未找到

```
 case 500:
```

```
 print("服务器内部错误")
```

```
 case _:
```

```
 print("未知状态码")
```

## ■ match-case 语句

- 多值匹配——用 | 合并模式
  - | 表示“或”，多个值共用一个分支

```
command = "quit"
```

```
match command:
```

```
 case "start" | "begin":
```

```
 print("程序启动")
```

```
 case "stop" | "quit" | "exit":
```

```
 print("程序退出") 输出: 程序退出
```

```
 case _:
```

```
 print("未知命令")
```

## ■ match-case 语句

- match 后面可以是任意合法的 Python 表达式

直接跟函数调用：

```
python

match int(input("请输入数字: ")):
 case 1:
 print("一")
 case 2:
 print("二")
 case _:
 print("其他")
```

跟运算表达式：

```
python

match a + b:
 case 0:
 print("和为零")
 case x if x > 0:
 print(f"和为正数: {x}")
 case x:
 print(f"和为负数: {x}")
```

## ■ match-case 语句

- case 内部仍然可以嵌套 if 等其他语句

```
op = input("请输入运算符: ")
a, b = 10, 3

match op:
 case "+":
 print(f"结果: {a + b}")
 case "-":
 print(f"结果: {a - b}")
 case "*" | "x":
 print(f"结果: {a * b}")
 case "/" | "÷":
 if b != 0:
 print(f"结果: {a / b}")
 else:
 print("除数不能为零")
 case _:
 print(f"不支持的运算符: {op}")
```

# 程序循环结构

---

Cycle structures of python procedure



# 循环结构

- **遍历循环**（确定次数循环），根据遍历结构中的元素个数确定循环次数，具体采用for语句实现
- **无限循环**（非确定次数循环），通过条件判断是否继续执行循环体，采用while语句实现

```
for <循环变量> in <遍历结构>:
 <语句块>
```

```
while <条件>:
 <语句块>
```



# 遍历循环: for语句

- 遍历结构可以是字符串、文件、组合数据类型或range()函数

| 用法     | 示例                                      | 说明               |
|--------|-----------------------------------------|------------------|
| 循环 N 次 | <code>for i in range(5):</code>         | 遍历 0, 1, 2, 3, 4 |
| 遍历字符串  | <code>for c in "Hello":</code>          | 逐个取出每个字符         |
| 遍历列表   | <code>for item in [1, 2, 3]:</code>     | 逐个取出每个元素         |
| 带下标遍历  | <code>for i, v in enumerate(ls):</code> | 同时获取下标和元素        |

# 遍历循环: for语句

- 遍历结构可以是字符串、文件、组合数据类型或range()函数

循环N次:

```
python

for i in range(5):
 print(i, end=" ")
输出: 0 1 2 3 4
```

遍历字符串:

```
python

for c in "Hello":
 print(c, end=" ")
输出: H e l l o
```

遍历列表:

```
python

for item in [10, 20, 30]:
 print(item, end=" ")
输出: 10 20 30
```

带下标遍历:

```
python

ls = ["苹果", "香蕉", "橘子"]
for i, v in enumerate(ls):
 print(f"第{i}个: {v}")
输出:
第0个: 苹果
第1个: 香蕉
第2个: 橘子
```

# 遍历循环: for语句

## ■ for-else

- else 里的代码，只在循环"完整跑完"时才执行；如果循环被 break 提前打断了，else 就跳过

for 循环变量 in 遍历结构:

语句块1

else:

语句块2      # 循环正常结束（未被 break 中断）时执行

# 遍历循环: for语句

## ■ for-else

```
names = ["李四", "王五", "赵六"] names = ["李四", "王五", "张三"]
```

```
for name in names:
 if name == "张三":
 print("找到张三了!")
 break
else:
 print("翻完了, 没有张三")
```

# 输出: 翻完了, 没有张三

```
for name in names:
 if name == "张三":
 print("找到张三了!")
 break
else:
 print("翻完了, 没有张三")
```

# 输出: 找到张三了!

⚠ 这次循环被 break 中断了, else 不执行



# 遍历循环: for语句

## ■ for-else

- 触发方式总结

| 循环结束方式                               | else 是否执行 |
|--------------------------------------|-----------|
| 正常跑完所有轮次                             | 执行        |
| 被 <code>break</code> 中断              | 不执行       |
| 循环体一次都没进入 (如 <code>range(0)</code> ) | 执行        |

⚠ 第三种情况如果遍历结构本身就是空的，循环体一次都不执行，但 `else` 照样会执行

# 遍历循环: for语句

## ■ 嵌套循环

- 循环里面再放一个循环，外层每执行一轮，内层完整跑一遍

```
for i in range(3): # 外层循环 3 次
 for j in range(4): # 内层循环 4 次
 print(f"({i},{j})", end=" ")
 print() # 每行结束换行
```

输出:

```
(0,0) (0,1) (0,2) (0,3)
(1,0) (1,1) (1,2) (1,3)
(2,0) (2,1) (2,2) (2,3)
```

**外层控制行，内层控制列**  
**总执行次数 = 外层次数 × 内层次数**

# 遍历循环: for语句

## ■ 嵌套循环

- 九九乘法表

$$1 \times 1 = 1$$

$$1 \times 2 = 2 \quad 2 \times 2 = 4$$

$$1 \times 3 = 3 \quad 2 \times 3 = 6 \quad 3 \times 3 = 9$$

$$1 \times 4 = 4 \quad 2 \times 4 = 8 \quad 3 \times 4 = 12 \quad 4 \times 4 = 16$$

...

$$1 \times 9 = 9 \quad 2 \times 9 = 18 \quad 3 \times 9 = 27 \quad \dots \quad 9 \times 9 = 81$$

如何输出三角形而不是正方形的乘法表？



# 遍历循环: for语句

## ■ 嵌套循环

- 九九乘法表

```
for i in range(1, 10):
 for j in range(1, i + 1):
 print(f"{j}×{i}={i*j}", end="\t")
 print()
```

关键在于内层的终止条件 `range(1, i+1)`，让每行列数随行号递增

## 无限循环: while语句

- 无限循环一直保持循环操作直到特定循环条件不被满足才结束，不需要提前确定循环次数。

```
while <条件>:
 <语句块>
```

- 不确定次数循环也有一种使用保留字else的扩展模式：

```
while <条件>:
 <语句块1>
else:
 <语句块2>
```

# 无限循环: while语句

## ■ while-else

- 逻辑和 for-else 完全一样：while 条件自然变为 False 正常结束时执行 else，被 break 中断则跳过 else
- 猜数字游戏：

```
secret = 42
chances = 3

while chances > 0:
 guess = int(input("猜一个数字: "))
 if guess == secret:
 print("恭喜你, 猜对了!")
 break
 chances -= 1
 print(f"猜错了, 还剩{chances}次机会")
else:
 print("机会用完了, 答案是42")
```

# 无限循环: while语句

## ■ while-else

- 猜数字游戏运行效果：

猜对了的情况——`break` 跳出，`else` 不执行：      没猜对的情况——循环正常结束，`else` 执行：

猜一个数字：10  
猜错了，还剩2次机会  
猜一个数字：42  
恭喜你，猜对了！

猜一个数字：10  
猜错了，还剩2次机会  
猜一个数字：20  
猜错了，还剩1次机会  
猜一个数字：30  
猜错了，还剩0次机会  
机会用完了，答案是42

# 循环保留字: break和continue

- 循环结构有两个辅助保留字：**break** 和 **continue**，用于辅助控制循环执行
- **break**：跳出**最内层** for 或 while 循环，脱离该循环后程序从循环后代码继续执行

```
for c in "PYTHON":
 if c == "H":
 break
 print(c, end=" ")
print("循环结束")
输出：
```

运行到 H 后循环直接结束，H、O、N 都不会被处理，程序从循环后的 print 继续执行

# 循环保留字: break和continue

- 循环结构有两个辅助保留字：**break** 和 **continue**，用于辅助控制循环执行
- **break**：跳出**最内层** for 或 while 循环，脱离该循环后程序从循环后代码继续执行

运行结果：

A: 1 2 3

B: 1 2 3

C: 1 2 3

```
for s in "ABC":
 print(s, end=": ") # 外层：打印字母
 for i in range(1, 6):
 if i == 4:
 break
 print(i, end=" ") # 内层：打印数字
 print()
```

数字没有打印到 5，说明内层循环被 **break** 提前结束；A、B、C 三个字母都出现了，说明外层循环不受影响。**break** 只跳出它所在的那一层

## 循环保留字: break和continue

- **continue** : 结束当前当次循环，即跳过循环体中下面尚未执行的语句，但不跳出当前循环
- 对于 while 循环，继续求解循环条件。而对于 for 循环，程序流程接着遍历循环列表

```
for i in range(1, 6):
 if i == 3:
 continue
 print(i, end=" ")
```

*i == 3 时执行 continue，本次的 print 被跳过，所以没有 3；但循环没有结束，接着取下一个值 4、5*

输出：

```
1 2 4 5
```

## 循环保留字: break和continue

- **continue** : 结束当前当次循环，即跳出循环体中下面尚未执行的语句，但不跳出当前循环
- 对于 while 循环，继续求解循环条件。而对于 for 循环，程序流程接着遍历循环列表

```
i = 0
while i < 5:
 i += 1
 if i == 3:
 continue
 print(i, end=" ")
```

输出:

1 2 4 5

continue 之后程序回到 while i < 5 重新判断条件

⚠ i += 1 必须放在 continue 之前；如果放在 print 之后，i == 3 时每次都跳过自增，i 永远是 3，程序陷入死循环





# 循环保留字: break和continue

## ■ continue 和 break 的区别

| 对比项    | break   | continue                       |
|--------|---------|--------------------------------|
| 作用     | 结束整个循环  | 只结束本次循环                        |
| 执行后跳转到 | 循环之后的代码 | 下一次循环（for 取下一个元素，while 重新判断条件） |
| 循环是否继续 | 否       | 是                              |

```
for c in "HELLO":
 if c == "L":
 continue
 print(c, end=" ")
```

输出：H E O

```
for c in "HELLO":
 if c == "L":
 break
 print(c, end=" ")
```

输出：H E

## 循环保留字: break和continue

- else 里的代码，只在循环"完整跑完"时才执行；如果循环被 break 提前打断了，else 就跳过
- continue 对else没有影响

```
for c in "HELLO":
 if c == "L":
 continue
 print(c, end=" ")
else:
 print("正常退出")
```

输出：H E O 正常退出

```
for c in "HELLO":
 if c == "L":
 break
 print(c, end=" ")
else:
 print("正常退出")
```

输出：H E

# 循环保留字: break和continue

## ■ 练习题

```
for c in "PROGRAM":
 if c == "O":
 continue
 if c == "A":
 break
 print(c, end=" ")
else:
 print("正常退出")
```

(1) 写出这段代码的运行结果。

(2) 如果把第 4 行的 "A" 改为 "X", 运行结果是什么?

(1) 输出 : P R G R ; (2) 输出 : P R G R A M 正常退出

# 循环保留字: break和continue

## ■ 练习题——百钱百鸡问题

- 百钱百鸡是我国古代数学家张丘建在《算经》一书中提出的数学问题
- 鸡翁一值钱五，鸡母一值钱三，鸡雏三值钱一。百钱买百鸡，问鸡翁、鸡母、鸡雏各几何？
- 翻译成现代文是：公鸡 5 元一只，母鸡 3 元一只，小鸡 1 元三只，用 100 块钱买一百只鸡，问公鸡、母鸡、小鸡各有多少只？
- 请使用穷举法以及嵌套循环结构，遍历所有可能的组合情况